

Diagnose before you regularize

Use training data to fit. Use validation data to choose. Open the test set once.

Regularization supplies a preference where the training data are silent. Validation chooses its strength.

1 • Keep the split contract clean

TRAIN

Fit parameters θ .

VALIDATE

Choose the recipe, checkpoint, and every knob.

TEST

Estimate final performance once, after choices are frozen.

2 • Read the curves before choosing a cure

Pattern

train low; val high
both high

Diagnosis

high variance
high bias

First move

regularize or add valid data
improve model, features,
or optimization
restore the validation-best
checkpoint
do not regularize harder

val rises late

overtraining

gap small; both poor

still underfit

Bias-variance lens

$$E\left[\left(\hat{f}(x) - f(x)\right)^2\right] = \underbrace{\left(E\left[\hat{f}(x)\right] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}\left(\hat{f}(x)\right)}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

Regularization usually trades a little more **bias** for less **variance**.
Validation tells you whether the trade helped.

3 • One dataset, four outcomes

Degree-9 fit

no regularizer
weight decay
early stopping
input jitter

train MSE

0.0000
-
-
-

validation MSE

1.70
0.065
0.061
0.048

Same ten measurements and same flexible model. Different preferences choose different solutions.

4 • Choose the intervention site

Site
objective
loop

Method
weight decay
early stopping

Knob
 λ
 t^*

Preference
smaller weights
simpler training
trajectory
valid nearby views

data

augmentation /
noise

strength

model
targets

dropout
label smoothing

p_{drop}
 ε

redundant features
finite confidence

Mixup and CutMix act on both the examples and their targets. Change one site at a time when you can.

5 • Weight decay: prefer smaller weights

Intuition: among equally good fits, prefer the one that does not need large coefficients.

$$\mathcal{J}(w) = \frac{1}{2n} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

$$w_{t+1} = \underbrace{(1 - \eta\lambda)w_t}_{\text{shrink}} - \underbrace{\eta g_t}_{\text{data step}}$$

Sweep λ on a log scale. Too little leaves a high-variance fit; too much underfits.

Optional detail: with adaptive optimizers, decoupled AdamW keeps the shrink separate from the adaptive gradient step.

6 • Early stopping: select the best checkpoint

Intuition: later updates can fit training quirks after the useful pattern has already been learned.

$$t^* = \underset{t}{\operatorname{argmin}} \mathcal{L}_{\text{val}}(w_t)$$

- 1 save when validation improves
- 2 stop after patience is exhausted
- 3 restore the saved best state

The last epoch is not automatically the selected model.

Practical order: diagnose, pick one site, sweep its knob on validation, and keep the simplest recipe that actually helps.

Operate each regularizer correctly

The mechanism matters: what changes during training, what stays fixed, and which knob validation chooses.

1 • Augmentation: teach valid variation

Intuition: show task-valid versions of each sample so the model cannot rely on one stored form.

$$A_{\tau(x,y)} = (g_{\tau(x)}, h_{\tau(y)})$$

For every transform, decide:

- KEEP** the label is unchanged
- UPDATE** move boxes, masks, keypoints, spans, or times
- REJECT** the transformed example is no longer valid

Input jitter reuses the target locally:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I), \quad (x_i, y_i) \rightarrow (x_i + \varepsilon, y_i)$$

σ sets the neighbourhood size. Train transforms are stochastic; validation and test stay deterministic. Audit real batches before training.

2 • Mixup and CutMix: constrain the space between samples

Why it can help: the prediction must change smoothly between examples instead of forming a sharp, memorized boundary around each one.

$$\lambda \sim \text{Beta}(\alpha, \alpha), \quad \tilde{x} = \lambda x_i + (1 - \lambda)x_j,$$

$$\tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

A 70/30 cat-dog blend contributes $-0.70 \log p_{\text{cat}} - 0.30 \log p_{\text{dog}}$.

Before using it: ask whether the mixed input makes sense, whether the mixed target says what you want, and whether untouched validation data improves.

α controls how strongly examples mix; λ is redrawn. CutMix uses the pasted area fraction for the soft target.

3 • Dropout: practise with missing features

Intuition: random feature outages during training reward backup routes, so one feature cannot depend on one fixed partner.

Let $q_{\text{keep}} = 1 - p_{\text{drop}}$:

$$m_i \sim \text{Bernoulli}(q_{\text{keep}}), \quad \tilde{h}_i = \frac{m_i h_i}{q_{\text{keep}}}$$

$$E[\tilde{h}_i] = h_i, \quad \text{Var}(\tilde{h}_i) = \left(\frac{1 - q_{\text{keep}}}{q_{\text{keep}}} \right) h_i^2$$

- TRAIN** fresh mask; keep survivors scaled by $\frac{1}{q_{\text{keep}}}$
- EVAL** all units on; no mask and no extra scaling

PyTorch's `p` is the drop probability and scaling is automatic. `model.eval()` changes dropout behaviour; `inference_mode()` changes autograd.

4 • Label smoothing: ask for confidence, not certainty

Intuition: one-hot cross-entropy keeps rewarding a larger correct-class logit gap. Smoothing gives that push a finite stopping point.

$$\tilde{y} = (1 - \varepsilon)y + \frac{\varepsilon}{K} \mathbf{1}, \quad \frac{\partial \mathcal{L}}{\partial z} = p - \tilde{y}$$

With $K = 5$ and $\varepsilon = 0.1$:

$$(0, 1, 0, 0, 0) \rightarrow (0.02, 0.92, 0.02, 0.02, 0.02)$$

The gradient stops pushing class b when $p_b = 0.92$. Against any wrong class j , the finite target gap is

$$m^* = z_b - z_j = \ln\left(\frac{0.92}{0.02}\right) \approx 3.83.$$

m^* is the desired correct-vs-wrong logit margin. Validate ε : smoothing can soften too much and can erase useful class-similarity structure.

5 • A disciplined recipe

- 1 fit a reproducible baseline and plot learning curves
- 2 add only task-valid augmentation
- 3 sweep weight decay on a log scale
- 4 add one targeted method only if the curves justify it
- 5 freeze the recipe and checkpoint; evaluate test once

Keep the training budget and validation protocol fixed during comparisons.

6 • Final audit

- ✓ Learning curves diagnose the problem.
- ✓ One intervention site and one named knob.
- ✓ Validation chose the recipe and checkpoint.
- ✓ The best checkpoint was restored.
- ✓ The test set remained untouched until the end.
- ✓ Seed, splits, schedule, and every knob were logged.

If you have not plotted learning curves, diagnose the problem before choosing a regularizer.

One-line map

- Weight decay** constrains parameters.
- Early stopping** constrains training time.
- Augmentation** expands valid training views.
- Mixup/CutMix** constrain behaviour between examples.
- Dropout** disrupts hidden feature pathways.
- Label smoothing** softens the confidence target.